

TREE PLANTING APP – MVP

Product, UX, Architecture & Implementation Brief

Status: MVP Architecture Baseline

Purpose: This document is the source of truth for implementing the first production-ready version of the application.

1. ROLE

You are the Lead Product Engineer, Software Architect, UX Engineer, and Technical Lead responsible for implementing this application.

Do not treat this as a throwaway prototype.

The product is an MVP, but the architecture must

be production-quality, maintainable, testable, secure, and extensible.

"MVP" means:

- fewer features
- fewer screens
- simpler workflows

It does NOT mean:

- hardcoded data
- fake backend logic
- insecure trust models
- duplicated business logic
- architecture that must be rewritten after MVP
- placeholder implementations presented as finished functionality

Build the smallest correct system.

2. PRODUCT CONCEPT

The application is a nature-focused tree planting platform.

Users plant real trees and report them through the application.

A tree report may contain:

- tree species
- planting date
- geographic location
- GPS coordinates
- photographs
- optional notes
- other evidence required by the verification system

The application records these trees, allows users to explore planting activity, tracks personal impact, and awards achievements for

meaningful milestones.

Google Play Games is NOT a game system for this application.

There are no games in the MVP.

Google Play Games is intended to provide the achievement infrastructure/platform integration.

The product loop is:

Plant → Report → Prove → Verify → Track →
Achieve → Share → Inspire

3. MVP PRODUCT PRINCIPLES

The application must optimize for five things:

1. Trust
2. Simplicity
3. Motivation
4. Personal progress

5. Community impact

The application should feel like a premium environmental product rather than a generic social network.

The visual identity should communicate:

- nature
- greenery
- sustainability
- calmness
- optimism
- organic growth

The current visual direction uses:

- leafy greens
- warm cream/off-white
- paper-like textures
- botanical illustrations
- soft rounded cards
- organic shapes

- restrained shadows
- premium typography
- subtle environmental motifs

Do not over-gamify the UI.

Achievements should feel meaningful rather than childish.

4. MVP USER JOURNEY

The primary user journey is:

text



Install



Welcome



Continue with Google



Authenticate



Complete minimal onboarding



Home



Report a Tree



Enter tree details



Provide proof



Review



Submit



Verification state



Tree recorded



Achievement evaluation



Achievement unlocked if criteria are met



User sees updated progress

Secondary journeys:

text



Home

- Explore Nearby
- Find Friends
- My Impact
- Achievements
- Profile

—

5. MVP SCOPE

Implement the following functional areas.

A. Authentication

Required:

- Continue with Google
- user account creation
- existing-user login
- persistent authenticated session
- logout
- account/profile identity

The architecture must allow additional authentication providers later, but Google is the MVP provider.

Do not create a custom username/password system unless there is a strong architectural reason.

6. USER PROFILE

Each user should have a persistent profile.

Minimum conceptual model:

text



User

- id
- displayName
- username / publicHandle
- email
- profileImage
- createdAt
- updatedAt
- status

Do not expose private account information publicly.

Separate:

- authentication identity
- internal user identity
- public profile

The system must not assume that the Google

account itself is the application's domain model.

7. TREE DOMAIN MODEL

A tree is a first-class domain entity.

Conceptually:

text



Tree

- id
- ownerId
- speciesId
- plantedAt
- latitude
- longitude
- locationAccuracy
- locationSource
- status
- verificationStatus
- createdAt

- updatedAt

Possible status values should be explicitly modeled rather than represented by arbitrary strings.

For example:

text



DRAFT
SUBMITTED
UNDER_REVIEW
VERIFIED
REJECTED
ARCHIVED

Do not award permanent achievements based purely on unverified claims unless the product requirements explicitly say so.

8. TREE SPECIES

Species should be represented independently from individual trees.

Conceptually:

text



TreeSpecies

- id
- commonName
- scientificName
- aliases
- metadata
- status

Do not hardcode species names throughout UI code.

The system must allow the species catalogue to grow.

9. TREE REPORTING WORKFLOW

The MVP reporting flow is:

Step 1 — Details

User selects:

- tree species
- planting date
- location

Optional:

- planting context
- notes

Step 2 — Proof

User provides:

- at least one photograph
- GPS/location evidence

The application should capture metadata needed for verification.

Step 3 — Review

Show a clear summary:

text



Species

Planting date

Location

Photos

Notes

The user confirms before submission.

Step 4 — Submission

The system creates the tree/report record.

Do not treat client-side submission as authoritative.

The server must validate:

- authenticated user
- required fields
- valid species
- valid coordinates
- acceptable date
- proof requirements
- upload references
- duplicate constraints where applicable

10. PHOTO STORAGE

Do not store large image binaries directly inside the primary relational database.

Use object storage.

The architecture should conceptually be:

text



Mobile App



Secure upload mechanism



Object Storage



Media Metadata



Tree

Media metadata should include appropriate fields such as:

text



TreeMedia

- id
- treeld
- storageKey

- mediaType
- capturedAt
- metadata
- createdAt

Do not trust arbitrary client-supplied storage paths.

Use server-authorized upload mechanisms.

11. LOCATION / GPS

Location is a core part of tree proof.

Store:

- latitude
- longitude
- accuracy
- source
- timestamp where available

Do not automatically expose exact coordinates publicly.

This is an important privacy and safety requirement.

Public map views may need coordinate obfuscation, clustering, rounding, or another privacy-preserving mechanism.

The owner may have access to more precise information than other users.

12. VERIFICATION

Verification is one of the most important architectural concerns.

The system must distinguish:

text



Reported

from

text



Verified

A user submitting a tree does not automatically make the claim trusted.

MVP verification can initially be relatively simple, but the architecture must support more sophisticated verification later.

Possible MVP states:

text



SUBMITTED

PENDING_VERIFICATION

VERIFIED

REJECTED

Design the domain so future verification methods can be added without rewriting the

Tree model.

Potential future evidence sources:

- GPS consistency
- photo metadata
- timestamps
- duplicate-photo detection
- location history
- human review
- organization verification
- community verification
- planting campaigns
- external integrations

Do not implement all of these in MVP.

Create the correct extension point.

13. ANTI-ABUSE / TRUST

Assume users may attempt to abuse the achievement system.

Examples:

- reporting fake trees
- submitting the same tree repeatedly
- submitting duplicate photos
- manipulating GPS
- creating multiple accounts
- rapidly generating reports

The MVP does not need an advanced fraud-detection engine.

However, the architecture must not make abuse impossible to address later.

At minimum:

- server-authoritative records
- authenticated requests
- validation
- rate limiting

- duplicate detection strategy
- verification state
- auditability
- immutable or controlled achievement awarding
- server-side achievement evaluation

Never rely solely on:

text



```
if treesCount >= 25:  
    unlockAchievement()
```

inside the client.

14. ACHIEVEMENT SYSTEM

Achievements are a first-class domain concept.

The achievement engine should be independent from the UI.

Conceptually:

text



AchievementDefinition

- id
- key
- name
- description
- category
- criteria
- icon
- active

And:

text



UserAchievement

- userId
- achievementId
- status

- progress
- unlockedAt
- metadata

The achievement system should support:

- progress tracking
- locked achievements
- unlocked achievements
- milestone achievements
- category filtering
- future achievement types

15. MVP ACHIEVEMENTS

Start with a small meaningful set.

Examples:

First Tree

Plant your first verified tree.

Tree Planter

Plant 10 verified trees.

Tree Planter II

Plant 25 verified trees.

Forest Builder

Plant 100 verified trees.

Biodiversity

Plant 5 different verified species.

Green Streak

Meet a defined reporting/planting streak condition.

Community Hero

Meet a defined community/social contribution condition.

Do not hardcode these directly into individual UI screens.

They must be definitions evaluated by an achievement subsystem.

16. ACHIEVEMENT EVALUATION

The server should be authoritative.

A conceptual flow:

text



Tree becomes VERIFIED



Domain event / transaction



Achievement evaluator



Evaluate relevant achievement criteria



Update UserAchievement



If newly unlocked:



Create achievement event



Sync with Google Play Games where applicable



Notify client

Do not require the mobile client to calculate whether an achievement should exist.

The client renders the state supplied by the backend/platform.

17. GOOGLE PLAY GAMES

Google Play Games should be treated as an external platform integration.

Do not make the core application dependent on Google Play Games being available for the application to function.

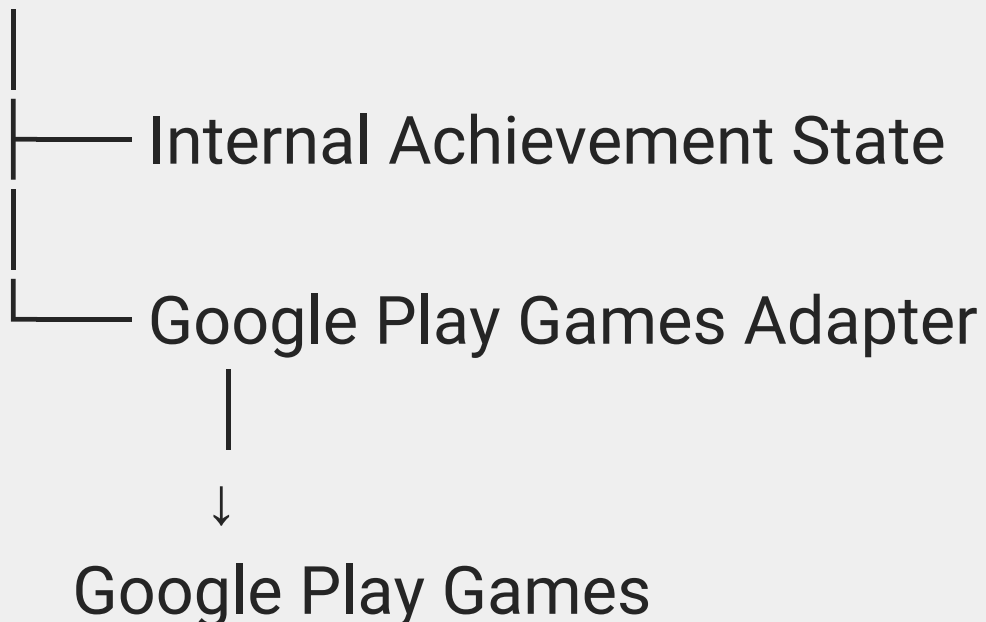
The core application must work independently.

Conceptually:

text



Application Achievement System



This separation is critical.

If Google Play Games is unavailable:

- tree reporting should still work
- achievements should still be tracked internally
- the user should still see achievements
- synchronization can retry later

Verify the current official Google Play Games Services APIs and authentication requirements before implementation. Do not rely on outdated SDK assumptions.

18. HOME DASHBOARD

The Home screen is the command center.
It should show:

Primary metric

Trees planted

Supporting metrics

- achievements
- friends
- communities or locations, depending on final MVP scope

Quick Actions

- Report a Tree
- Explore Nearby
- Find Friends
- View My Impact

Progress

Show the user's next relevant achievement.

Example:

text



Tree Planter II

12 / 25

13 trees to go

The Home screen should not become an analytics dashboard overloaded with information.

19. QUICK ACTIONS

The Quick Actions UI can be a bottom sheet or equivalent interaction.

It should not necessarily be a permanent

navigation destination.

Actions:

text



Report a Tree

Explore Nearby

Find Friends

View My Impact

The underlying destinations are real application features.

The Quick Actions component is simply a convenient launcher.

20. EXPLORE NEARBY

MVP should provide a map-based discovery experience.

Show:

- nearby verified trees
- user location where permitted
- tree markers/clusters
- search
- basic filtering

Potential filters:

- all trees
- my trees
- friends
- recently planted

Do not expose exact private coordinates unnecessarily.

Tapping a tree should open Tree Detail.

21. TREE DETAIL

A tree detail page should show:

- species
- planting date
- approximate location
- planter/public identity where appropriate
- photograph
- verification state
- relevant metadata

Do not expose sensitive verification data.

22. FIND FRIENDS

MVP social functionality should remain intentionally small.

Provide:

- search
- suggested users
- add friend
- pending request
- accepted friend state

Avoid building a full social network.

No MVP requirement for:

- chat
- direct messaging
- comments
- complex social feeds

The architecture can support these later but should not implement them now.

23. FRIEND PROFILE

A public profile can show:

- display name
- avatar
- tree count
- achievement count
- public milestones

Do not expose:

- email
- private account data
- exact private tree coordinates
- security/account information

24. IMPACT

MVP should support two views:

Personal

Show:

- verified trees planted
- species count
- locations
- planting history
- achievement progress
- carefully defined environmental estimates

Global

Show:

- total verified trees
- participating users

- community/global totals
- map/visual representation where practical

Do not invent environmental claims.

Any metric such as CO₂ impact must have an explicit documented calculation methodology.

If the methodology is not ready, label the metric appropriately or omit it.

Never present scientifically uncertain estimates as precise facts.

25. MY TREES

The user should have a personal tree journal.

Each tree can show:

- photograph
- species

- planting date
- location
- verification status

This should become the user's personal history of real-world planting.

26. PROFILE

Profile should contain:

text



User identity

Stats

My Trees

My Impact

Achievements

Friends

Google Play Games connection status

Settings

Privacy

Help / Support

Logout

Keep the MVP profile simple.

27. SETTINGS

MVP settings should include only necessary controls.

Potentially:

- account
- privacy
- notification preferences
- location preferences
- connected services

- logout
- delete account

Account deletion must be designed properly rather than simply deleting the local session.

28. NAVIGATION

Recommended primary navigation:

text



HOME

MAP / EXPLORE

+

ACHIEVEMENTS

PROFILE

The central + launches Quick Actions.

The primary navigation should remain stable across the application.

29. SCREEN INVENTORY

MVP target:

Authentication

1. Welcome
2. Google authentication
3. Minimal onboarding / permissions

Core

4. Home
2. Quick Actions
3. Report Tree — Details

4. Report Tree — Proof
5. Report Tree — Review
6. Report Success

Discovery

10. Explore Nearby
2. Tree Detail

Social

12. Find Friends
2. Friend Profile

Impact

14. My Impact — Personal
2. My Impact — Global

Achievements

16. Achievements

2. Achievement Detail

Account

18. Profile

2. My Trees

3. Settings

Some of these may be states/components rather than separate navigation destinations.

Do not create screens merely because they exist in this list.

Use proper UX judgment.

30. UX STATE REQUIREMENTS

Every important workflow must account for:

- loading
- empty
- success
- failure
- offline/intermittent connectivity
- permission denied
- authentication failure
- invalid input
- upload failure
- submission failure
- verification pending
- verification rejected
- already completed
- unavailable external service

Do not design only the happy path.

31. OFFLINE / NETWORK BEHAVIOR

The application should degrade gracefully.

At minimum:

- cached user/profile data where appropriate
- clear offline state
- safe retry behavior
- no duplicate tree submissions due to retries
- idempotent submission semantics where appropriate

A network timeout must not result in:

text



Tree A

Tree A

Tree A

being accidentally created.

32. BACKEND PRINCIPLES

The backend must be authoritative for:

- users
- trees
- verification state
- achievement state
- social relationships
- aggregate statistics
- permissions
- business rules

Do not put important business rules only in the mobile application.

The mobile application is an untrusted client.

33. DOMAIN SEPARATION

Keep the following concepts separate:

text



Authentication

User

Tree

Tree Species

Tree Evidence / Media

Verification

Achievement

Achievement Progress

Friendship

Impact

Notification

External Integrations

Do not create one giant User/Tree service containing every feature.

Use clear domain boundaries even if the initial deployment is a modular monolith.

34. RECOMMENDED MVP ARCHITECTURE

Unless there is a compelling reason otherwise:

Mobile

Use a maintainable layered architecture:

text



Presentation



Application / ViewModel



Domain



Data

Keep UI code free of core business rules.

Backend

Prefer:

text



API



Application Services



Domain



Persistence

with infrastructure integrations separated.

For MVP, a modular monolith is preferable to premature microservices.

Do NOT create microservices simply to make the architecture look sophisticated.

35. MODULAR BACKEND STRUCTURE

A reasonable conceptual structure:

text



/auth

/users

/trees

/species

/media

/verification

/achievements

/friends

/impact

/notifications

Each module should own its domain logic.

Shared infrastructure should remain genuinely shared.

Do not allow arbitrary cross-module database manipulation.

36. API DESIGN

Use consistent APIs.

For example:

text



POST /auth/...

GET /me

GET /trees

POST /trees

GET /trees/{id}

POST /trees/{id}/submit

GET /species

GET /achievements

GET /me/achievements

GET /explore/nearby

GET /friends

POST /friends/requests

DELETE /friends/requests/{id}

GET /me/impact

GET /impact/global

These are conceptual examples.

Adapt them to the actual framework and architecture.

Do not blindly copy them.

37. DATABASE PRINCIPLES

Use proper relational modeling where relationships and consistency matter.

Important constraints should exist at the database level where practical.

Examples:

- unique user identity
- unique achievement unlock
- valid foreign keys
- appropriate indexes
- timestamps
- controlled statuses

Do not rely solely on application-level validation for data integrity.

38. AUDITABILITY

Important actions should be traceable.

At minimum, the architecture should allow us to answer:

text



Who submitted this tree?

When?

What evidence was supplied?

What was the verification state?

When did it become verified?

Why was an achievement awarded?

When was it awarded?

This is especially important because the achievement system represents claims about real-world activity.

39. SECURITY

Treat the mobile client as hostile/untrusted.

Implement appropriate:

- authentication
- authorization
- server-side validation
- secure media uploads
- rate limiting
- input validation
- access controls
- privacy controls
- secrets management
- secure storage
- logging without sensitive information

Never trust:

- user IDs supplied by clients

- achievement unlock requests
 - tree counts supplied by clients
 - verification status supplied by clients
 - arbitrary storage paths
 - coordinates without validation
-

40. PRIVACY

This application deals with location data. Privacy must therefore be designed into the architecture.

Important principles:

- exact GPS coordinates are sensitive
- public views should not automatically expose exact coordinates
- users should understand why location is

collected

- permission should be requested contextually
- private data must not leak through APIs
- authorization must be enforced server-side

Do not log precise user coordinates unnecessarily.

Do not include sensitive location data in analytics events.

41. OBSERVABILITY

Implement proper production observability from the beginning.

At minimum:

- structured logging
- error tracking

- request correlation IDs
- basic metrics
- important domain event logging

Avoid logging:

- authentication secrets
- access tokens
- private user data
- unnecessary exact GPS coordinates
- private photographs

42. TESTING

Testing is part of implementation, not a future task.

Minimum expectations:

Unit tests

For:

- achievement criteria
- verification rules
- tree validation
- impact calculations
- permission/business rules

Integration tests

For:

- authentication
- tree creation
- media workflow
- verification transition
- achievement awarding
- friendship operations

API tests

For:

- authorization
- validation
- error states
- idempotency

UI tests

For critical flows:

text



Login

Report Tree

Submit Tree

Achievement unlock

Explore

Friend request

43. ACHIEVEMENT ENGINE TESTING

This deserves special attention.

Test:

text



1 tree → First Tree
9 trees → not Tree Planter
10 trees → Tree Planter
24 trees → not Tree Planter II
25 trees → Tree Planter II

Also test:

text



same tree submitted twice
same event processed twice
network retry

achievement already unlocked
verification revoked/rejected
multiple achievements unlocked
simultaneously

The system must be idempotent.

An achievement must not be awarded twice simply because an event was processed twice.

44. DESIGN SYSTEM

Create reusable components rather than screen-specific styling.

Core components:

- buttons
- cards
- navigation
- bottom sheets

- input fields
- dropdowns
- segmented controls
- badges
- achievement cards
- progress bars
- map markers
- avatars
- image cards
- empty states
- error states
- loading states
- confirmation dialogs

Design tokens should define:

- typography
- spacing
- radii
- shadows
- colors

- iconography
- elevation
- motion

The paper/leaf aesthetic should be implemented as a coherent system.

Do not manually style every screen independently.

45. VISUAL DIRECTION

Use the existing design direction as inspiration:

Nature × Premium × Paper

Primary visual language:

- botanical green
- deep forest green
- warm cream

- muted earth tones
- subtle paper grain
- botanical illustrations
- organic curves
- restrained gradients
- clean modern typography

Avoid:

- childish cartoon styling
- excessive neon
- generic SaaS blue
- overly gamified UI
- clutter
- excessive glassmorphism
- stock photography dominating the UI

Photography should primarily represent actual planted trees/proof.

46. IMPORTANT PRODUCT DISTINCTION

This application is NOT:

- a game
- a generic social network
- a tree photo gallery
- a leaderboard app
- a carbon calculator

It is:

A verified real-world tree planting platform with personal progress, achievements, discovery, and community impact.

The real-world action is the product.

The digital experience exists to:

record it → verify it → celebrate it → connect it
→ amplify it.

47. MVP EXCLUSIONS

Do not implement unless explicitly approved later:

- in-app games
- virtual currency
- marketplace
- donations
- subscriptions
- advertisements
- chat
- direct messaging
- complex feeds
- comments
- advanced leaderboards
- organization management

- enterprise dashboards
- tree adoption marketplace
- blockchain/NFT functionality
- complex AI verification
- satellite verification
- advanced fraud detection

Architect for future expansion where reasonable, but do not build these now.

48. FUTURE EXTENSIBILITY

The architecture should make it possible to add later:

text



Organizations

Schools

Companies

Planting campaigns
Community challenges
Leaderboards
Advanced verification
AI-assisted photo verification
Environmental organizations
Partner integrations
Rewards
More authentication providers
More achievement providers
Web dashboard
Admin console

Do not prematurely implement them.

49. ADMIN / OPERATIONS CONSIDERATION

Even if the full admin UI is outside MVP, the

backend must allow authorized operators to:

- inspect tree reports
- inspect verification state
- review evidence
- reject invalid reports
- correct data through controlled operations
- inspect achievement state
- investigate abuse

Do not create a system where the development team must directly edit production database rows to resolve ordinary operational issues.

If the full admin UI is not included in MVP, create the appropriate service/API boundaries so it can be added safely.

50. IMPLEMENTATION RULES

Before writing substantial code:

1. Inspect the repository.
2. Understand the existing stack.
3. Identify existing architecture.
4. Identify reusable infrastructure.
5. Identify contradictions or technical debt.
6. Produce an implementation plan.
7. Validate the plan against this document.
8. Only then begin implementation.

Do not rewrite working infrastructure without justification.

Do not introduce technologies merely because they are fashionable.

Do not create abstractions without a real purpose.

Do not hardcode domain data that belongs in configuration/database/domain definitions.

Do not duplicate business rules between client and server.

Do not hide incomplete functionality behind fake success states.

51. DEFINITION OF DONE

A feature is NOT complete merely because its screen exists.

A feature is complete when:

- UI exists
- backend/API exists where required
- persistence exists
- validation exists
- authorization exists
- error states exist
- loading states exist
- empty states exist
- tests exist

- analytics/observability are appropriate
 - security/privacy have been considered
 - business logic is server-authoritative where required
 - the feature works end-to-end
-

52. AGENT BEHAVIOR

Be critical.

If this specification contains a contradiction, identify it.

If an architectural decision is unsafe, say so.

If a requirement is technically incorrect, challenge it.

If a requirement cannot be implemented reliably as written, stop and explain the issue before silently implementing an inferior workaround.

Do not say:

"That's fine for MVP"

when the underlying decision creates architectural debt, security problems, data integrity problems, or a rewrite later.

Distinguish between:

text



Acceptable MVP simplification

and

text



Unacceptable architectural shortcut

Examples of acceptable MVP simplification:

- simple verification workflow
- small achievement catalogue
- basic friend discovery

- limited map filters
- basic impact calculations

Examples of unacceptable shortcuts:

- client-controlled achievement unlocks
- client-controlled tree counts
- storing everything in one database table
- exposing private GPS coordinates
- fake backend APIs
- hardcoded user data
- no verification state
- no idempotency
- no authorization
- putting all business logic in UI
- architecture that cannot support Google Play Games integration cleanly

53. REQUIRED IMPLEMENTATION PHASES

Implement in phases.

Phase 0 — Architecture Audit

Before coding:

- inspect repository
- document current stack
- identify architecture
- identify risks
- identify missing infrastructure
- produce target architecture

Do not start blindly.

Phase 1 — Foundation

Implement:

- authentication
- user model
- database foundation
- API foundation
- secure configuration
- navigation
- design system foundation
- error handling
- observability

Phase 2 – Tree Reporting

Implement:

- species
- tree entity
- media
- location
- report workflow
- validation

- submission
- verification states

Phase 3 — Achievements

Implement:

- achievement definitions
- user achievement state
- progress
- achievement evaluator
- idempotency
- Google Play Games integration adapter

Phase 4 — Discovery

Implement:

- nearby map
- tree detail
- privacy-aware location display

- search/filtering

Phase 5 — Social

Implement:

- friend search
- friend requests
- friend profiles

Phase 6 — Impact

Implement:

- personal impact
- global impact
- documented calculations

Phase 7 — Profile / Settings

Implement:

- profile
- My Trees
- settings
- privacy
- connected services
- account lifecycle

Phase 8 — Hardening

Perform:

- security review
- authorization review
- data integrity review
- performance review
- offline/retry review
- achievement abuse review
- privacy review
- accessibility review
- test coverage review

- production readiness review

54. FIRST TASK

Do NOT immediately start implementing the UI.
First:

A. Inspect the entire existing repository.

B. Determine:

- framework
- language
- application structure
- backend
- database
- authentication
- storage

- navigation
- state management
- existing design system
- testing infrastructure
- CI/CD
- environment management
- deployment architecture

C. Compare the current architecture against this specification.

D. Identify:

- blockers
- architectural risks
- security risks
- data-model problems
- missing infrastructure
- unnecessary complexity
- contradictions

- technical debt

E. Produce:

MVP Architecture Review & Implementation Plan

The report must explicitly distinguish:

text



KEEP

MODIFY

REMOVE

ADD

DEFER

for existing components.

Do not begin large-scale implementation until this architecture review has been completed.

55. FINAL PRINCIPLE

Build the MVP as if it will succeed.

The first release may have only a few thousand users.

The architecture should nevertheless be capable of evolving into a much larger platform without replacing the fundamental domain model.

The central invariant is:

Jobs are not the product here; verified real-world planting activity is the product.

Everything else exists around that core:

text



REAL WORLD ACTION



TREE



PROOF



VERIFICATION



Preserve this model throughout implementation.